PROJECT OVERVIEW

Kavach Sovereign Workbench

An air-gapped, multi-model agentic AI workbench that runs on one mid-range GPU — and proves it never talks to anything outside the plant.

Problem statement	SIH **26117** — Smart Automation
Client	MRPL — Mangalore Refinery & Petrochemicals Ltd.
Reference architecture	KVC-ARCH-001 Rev A
Stack	Python 3.11+, FastAPI, SQLite, llama.cpp, numpy, Docker
Codebase	~27,300 lines across services / scripts / tests
Test suite	**789 passed**, 0 failed (67s) · ruff clean
Build status	All 7 phases code-complete; 6 real-model checkpoints outstanding
Document date	10 September 2026

> **The one rule that shapes every file**
>
> **Never let a degraded thing look like a working thing.** Every subsystem that cannot do its real job says so, loudly, in-band — it does not quietly do something weaker. This is a sovereignty product; a reassuring number nobody can defend is worse than an honest gap.

1. What this is, and why it exists

A refinery has documents it legally cannot send to a cloud LLM: standard operating procedures, P&IDs, inspection and hydrotest reports, approval notes, handwritten annotations on drawings. It also has no budget for an H100 cluster sitting in a rack at the plant. Between those two facts sits a gap that most "enterprise AI" answers fall straight through.

Kavach is the middle path. It is a workbench where a plant engineer asks questions, generates code, summarises documents and runs multi-step agentic tasks against **local open-weight models on a single 8–16 GB graphics card**, behind a hard sovereignty boundary, with a tamper-evident record of everything that happened.

Three things make it different from "a chatbot with a local model":

- **It routes.** A 12 GB card cannot hold four models at once, so a router decides per request which model to use, and a VRAM warden decides what gets evicted to make room. Every decision keeps its arithmetic and is shown to the user.
- **It is agentic, with brakes.** Plan → execute → observe → critique → replan, with terminal budgets, approval gates that survive a process restart, and a critic that fails closed.
- **It proves its own containment.** A default-DROP firewall, a sentinel reading real kernel packet counters, and a canary that deliberately attempts egress so the block can be watched working — because a dashboard that always reads zero is indistinguishable from a dashboard that is broken.

2. Architecture — seven layers, all built

Layer	Name	What it actually contains	Status
L0	Sovereignty	nftables default-DROP egress policy with named counters; a **sentinel** reading real kernel TX counters (`/proc/net/dev` on Linux, `GetIfTable2` on Windows); a **canary** that deliberately attempts egress to prove enforcement; a classification → capability policy engine; a SHA-256 tamper-evident audit chain over SQLite	DONE
L1	Model fabric	YAML model registry with digest verification at boot; a **VRAM warden** doing admission control, LRU eviction and swap costing; mock and llama.cpp backends; hot reload with no restart	DONE
L2	Router	Four-stage cascade — hard constraint elimination, kNN task classifier over labelled prototypes, normalised utility scoring, verifier with a bounded one-tier escalation; plus an offline eval harness	DONE
L3	Knowledge	The ingest fork (native text layer vs. a picture of text), page-exact chunking, BM25 + dense hybrid retrieval fused by Reciprocal Rank Fusion, clearance filtering, mandatory resolvable citations	DONE
L4	Tools + sandbox	Manifest-declared tools, a workspace path jail, `code.exec` in a Docker container with `--network=none`, read-only rootfs and seccomp, GBNF grammars generated from JSON schema so a malformed tool call is unreachable	DONE
L5	Agent core	plan / execute / observe / critic / replan, terminal budgets, approval gates, resumable runs persisted in SQLite	DONE
L6	UI + harness	Five panels behind a sidebar (overview, sovereignty, router, runs, corpus), a live SSE run stream, an offline-install checker, and an auto-generated run sheet	DONE

Everything is reachable three ways: the `kavach` CLI (about thirty subcommands), a FastAPI gateway on `127.0.0.1:8420`, and a browser panel served same-origin by that same gateway. There is no fourth path and no

separate frontend deployment — the UI ships inside the Python wheel, because a panel that has to be deployed separately is a panel that is missing on the venue box.

Repository map

```
config/            profiles.yaml (TINY/LOW/MID/HIGH/ARC/FAST) - models.yaml - policy.yaml
                   router.yaml - prototypes.yaml - knowledge.yaml - eval_tasks.yaml
services/kavach/
  config.py        profile + settings loading, all from local files
  types.py         Label, SideEffect, TaskClass, Run/Step/Budget vocabulary
  store/           SQLite schema + access (runs, steps, decisions, audit, egress)
  audit/chain.py   the tamper-evident hash chain
  sovereignty/     sentinel.py - netcounters.py - canary.py - policy.py
  runtime/         registry.py - warden.py - backends/{base,mock,llamacpp}.py
  router/          constraints.py - classifier.py - scoring.py - verifier.py - cascade.py
  tools/           workspace.py - registry.py - executor.py - grammar.py
                   manifests/ (calc, code.exec, fs.read, fs.list, fs.write, kb.search, doc.write)
                   impl/ (calc.py, fs.py, sandbox.py, kb.py, deliverables.py)
  agent/           planner.py - critic.py - loop.py - state.py - broker.py - references.py
  knowledge/       extract.py - chunk.py - store.py - index.py - retrieval.py
                   citations.py - answer.py
  api/             app.py + routes_{models,router,tools,runs,kb,stream}.py
  ui/              index.html - app.css - charts.js - app.js - theme.js   (no build step)
  cli.py           kavach doctor|serve|audit|canary|policy|models|route|tools|run|runs|kb
deploy/            nftables.conf - Dockerfile.gateway - Dockerfile.sandbox - seccomp-strict.json
scripts/           tamper_demo.py - provenance_demo.py - stage_model.py - eval_harness.py
                   swap_penalty_demo.py - sandbox_demo.py - knowledge_demo.py
                   offline_check.py - run_sheet.py - prove_sandbox.py - generate_mrpl_corpus.py
tests/             789 tests, most of them trying to break something
MRPL_DEMO_CORPUS/  native_pdfs - scanned_reports - pids - approval_notes - handwritten
```

3. The design principle, applied

"Never let a degraded thing look like a working thing" is not a slogan in this codebase; it is a checklist every subsystem was built against. These are the established instances — anything new follows the same pattern.

Subsystem	What it does when it cannot do the real thing
Canary	Returns `INCONCLUSIVE` on a timeout, never "blocked". A dropped packet and an unplugged cable are indistinguishable from inside the box
Sentinel	`DEGRADED` only when no counter source can be read at all — never a reassuring `0`
Sentinel	`MEASURED, NOT ENFORCED` when there is no nftables. Counting what left the box is not a kernel refusing to carry it, and `blocked` is then our own policy denials
Sentinel	An interface new since the baseline, or whose counter reset, is counted from boot and named in `detail`. Over-reporting can cry wolf; a silent `max(0, …)` reads zero forever
Model registry	`SIMULATED` provenance is a distinct value from `VERIFIED`, surfaced everywhere it appears
Mock backend	Every generation is stamped `[SIMULATED OUTPUT]`
Router	`quality_provenance: "seeded"` on every decision until the eval harness has actually run
Eval harness	**Refuses** `--quality` on the mock backend rather than stamping `quality_measured: true` on stub output
`code.exec`	**Refuses** without Docker; the dev fallback is opt-in and stamps `sandboxed: false`
Classifier	The keyword fallback reports `degraded: true` and prints `ACCURACY IS DEGRADED`
Critic	Fails **closed** — an unjudgeable step gets a human, not a silent pass
Ingest fork	A page nothing can read is `UNREADABLE`, listed by page number — never an empty chunk
Corpus coverage	Every search over a corpus with gaps repeats the gap count, in-band with the results
KB index	The dense arm switches **off** when the resident embedder is not the one that built the index; the index reports `STALE` rather than answering from the corpus it used to have
`kb.ask`	**Refuses** when nothing was retrieved. No context means no answer, not the model's recollection
Citations	Resolve, or the answer is not returned; `out_of_context` is its own verdict
SSE stream	A subscriber that overflows gets a `dropped` event carrying the count, never a silent truncation; `stream_end` is always sent even when the run thread died, because a quiet socket is not a thinking model
UI	The panel says `polling` when SSE is unavailable rather than looking live; every degraded subsystem hoists its reason into a banner at the top of the page
Run sheet	Opens with what was **not** proved, and refuses to fire the canary to improve its own numbers

4. The three claims it can defend

4.1 The audit log is tamper-evident

Every model call, tool invocation, file read, routing decision and approval writes one record. Each record carries the SHA-256 of its predecessor, so the log is a chain. SQLite append-only triggers stop casual edits; the hash chain catches an attacker who drops the triggers. The digest covers the canonical JSON of the record's own fields *including* `prev_sha256`, so a record cannot be re-pointed at a different predecessor without changing its own digest and every digest after it. Verification reports the **first** broken link, because everything after it is untrustworthy anyway.

`scripts/tamper_demo.py` breaks it five ways on purpose — an `UPDATE` (refused by trigger), a rewrite after dropping the triggers, a deleted record leaving a sequence gap, and a forged predecessor hash. This is the script to run when a judge asks "*how do you know the log hasn't been edited?*"

4.2 The policy engine governs tools it has never seen

Because it reasons over a declared **side-effect class** — READ, COMPUTE, EXECUTE, WRITE, DELETE — and a document **label**, not a tool name. Labels are PUBLIC → INTERNAL → CONFIDENTIAL → RESTRICTED and are inherited: document → chunk → run, where a run's effective label is the **maximum** of everything it has read. A RESTRICTED run cannot write outside the workspace at all (a hard refusal, visible in the demo) and cannot be routed to an unverified model. DELETE and FINAL_DELIVERABLE always stop for a human regardless of label.

4.3 The monitor is honest about what it cannot prove

On a box with no firewall the canary returns `INCONCLUSIVE`, not a green tick. That honesty is what makes the half it *does* prove worth believing — and it is the single property most likely to survive hostile questioning.

5. How the router works

A four-stage cascade. `decide()` is side-effect free, so the explainer panel can ask "what would you do?" without changing residency.

Stage	What happens
1. Constraints	Hard eliminations, with the reason kept for every eliminated candidate: unverified or mismatched weights, no vision tower when the request has images, no tool support when tools are needed, context window too short, will not fit in VRAM, above the policy tier for this label
2. Classifier	Embed the prompt, k-nearest-neighbour (k=5) against labelled prototypes into one of seven task classes: `code_generation`, `doc_summary`, `visual_doc`, `numerical_calc`, `official_drafting`, `kb_qa`, `agentic_multi`. Below 0.35 confidence the request is unclassified and falls back to the planner tier rather than guessing a specialist. **Improving the router means adding examples, not retraining anything.**
3. Scoring	`score = 1.00·quality + 0.35·confidence_fit − 0.15·latency − 0.20·swap_cost − 0.10·vram_pressure`, every term normalised 0–1
4. Verifier	Post-generation checks — minimum output length, an inline citation on `kb_qa` and `doc_summary`, parseable code on `code_generation`. A failure escalates the same prompt one tier up. **Once.** Never a loop

*Worked example on the MID profile with qwen3-8b resident and coder-7b cold: coder scores 0.88 − 0.095 − 0.20 − 0.047 = **0.538** and wins; the resident 8b scores 0.68 − 0.106 − 0.00 − 0.055 = 0.519. A twenty-point quality gap beats a 4.2-second reload; a five-point gap does not.*

6. How the knowledge plane works

The ingest fork is one decision, made per page, that everything downstream depends on: **does this page carry its own text, or is it a picture of text?** A native PDF page is read exactly — the characters in the index are the characters in the file. A scanned page has to be transcribed by something, and whatever transcribes it can be wrong. Those two are not interchangeable, so they are not recorded as the same thing.

1. Text layer at or above `min_native_chars` → **NATIVE**
2. A real OCR engine on this box → **OCR**
3. A real vision model, not a simulated one → **VLM**
4. The mock backend plus an explicit opt-in → **SIMULATED** (stamped everywhere, excluded from grounded answers, refused as evidence)
5. Nothing → **UNREADABLE**

Step 5 is the one that matters. Returning `""` for an unreadable page would let it join the index as an empty document contributing nothing, and the corpus would report eleven documents ingested with no asterisk. So an unreadable page is recorded as unreadable, counted, listed by page number, and surfaced on every search that touches its document.

Retrieval runs two arms — BM25 and dense vectors — and fuses them by **rank** (Reciprocal Rank Fusion), not by a weighted score sum. Clearance filtering is applied *after* fusion and the number of withheld chunks is returned: silently returning five hits instead of six teaches the user their query was weak, while saying "*2 chunks withheld: role 'operator' is cleared to INTERNAL*" teaches them the truth. Coverage travels with the result, because the only moment anyone will look at a corpus gap is when the answer is in front of them.

7. How the agent loop works

An agent is not a chat model with tools bolted on; it is a loop with state, budgets and a way to notice it has failed. Three properties the loop exists to guarantee:

> **Budgets are terminal**
>
> Exhausted means stop and report. It never means loop again, and it never means "one more try". A run that cannot finish inside its budget produces a partial result and says so.

> **Approval pauses, it does not block**
>
> A step needing sign-off parks the run in `AWAITING_APPROVAL` and returns. The process can exit; the run resumes later from SQLite. An approval gate that only holds while the process is up is theatre.

> **Failure replans once per step, at most twice per run**
>
> The critic's verdict feeds the re-planner, which amends the remaining steps. Bounded, because an unbounded replan loop on a 12 GB card is a demo watching a model change its mind.

Every tool invocation goes through one chokepoint, `ToolExecutor.invoke`, in a fixed order: resolve the tool, validate arguments against the declared schema, inspect for workspace escapes, apply the policy gate, **audit the intent**, execute, audit the outcome. Auditing the intent before the effect matters more than it looks — if the record were written only on success, a tool that hung or killed the process would leave no trace, and "no trace" is indistinguishable from "never happened" to an auditor.

8. Deliberate deviations from the reference architecture

Each of these was a considered decision with a failure mode behind it. They should not be "fixed" back without reading why.

#	Deviation	Why
1	Scoring terms are normalised 0–1	The doc's raw-seconds formula makes the latency term roughly ten times the quality term, so the router silently ranks by *speed* while every individual decision still looks plausible
2	`w_swap` is 0.20, not 0.30	Normalisation changed what the number means. At 0.30 a reload overrides a 0.30 quality gap, which is not "a few points" — it is enough to keep a materially worse model resident
3	Latency counts generation only	The doc's formula has `load_ms` in *both* the latency and the swap term, double-charging cold models. One cost, one term
4	HIGH profile `reserve_gb` is 1.6, not 2.2; the reranker is not pinned	At 2.2 the KV-cache charge double-counts, the second swap slot is squeezed out, and vision silently evicts text — defeating the entire point of the HIGH profile
5	TINY budgets against `ram_gb`	`cpu_only` means `vram_gb: 0.0`, so the contingency profile could otherwise load nothing — breaking the one profile whose whole job is still working when the GPU does not
6	No Qdrant. `knowledge/index.py` is numpy over a memory-mapped matrix	On one air-gapped box with a few thousand chunks that is sub-millisecond. A second daemon is another thing to install offline, keep alive and back up — and a second source of truth for "what is in the corpus" that can drift from the SQLite rows citations resolve against. Two answers to that question is exactly how a citation stops resolving
7	Retrieval fuses by rank (RRF), not a weighted score sum	Same failure as #1: BM25 is unbounded and corpus-dependent, cosine is 0–1, so a weighted sum silently becomes "BM25 only" on a corpus full of tag numbers — which is every refinery corpus
8	No frontend framework — three hand-written files	A bundle is a dependency tree to stage offline, a lockfile to audit and a build to reproduce on a venue machine, all to render four polling panels. `offline_check.py` can assert "zero absolute URLs" over three files in a way it could never assert over a minified vendor chunk
9	A chunk never spans a page boundary, whatever `target_chars` says	Costs a little recall at page joins; buys the only thing that makes a citation checkable. `[doc p.3]` on a chunk that is half on p.4 sends a reader to a page where half the quoted text is not

9. Bugs found the hard way

All of these now have regression tests. They are listed because each one is a class of mistake, not a one-off.

Bug	What it taught
`calc` had a code-execution hole	`sympy.sympify()` *evaluates* its input; `__import__('os').system(…)` ran. `calc` is COMPUTE, so it has **no sandbox**. Fixed with an AST allowlist gating both engines — any new tool taking a model-authored expression needs the same treatment
Warden / backend divergence	App shutdown cleared the backend while the warden, a process-level singleton, kept claiming residency. Shutdown now tears down *through* the warden; boot re-pins; `admit()` reconciles and audits

`stage_model.py` destroyed every comment in `models.yaml`	A `yaml.safe_dump` round-trip erases comments. Config writers do **surgical line edits**
A `StaticFiles` mount at `/` swallows the whole API	Starlette matches mounts in registration order alongside routes, so the UI mount is the **last** thing `create_app()` does
FastAPI request models must be at module scope	`from __future__ import annotations` plus a model defined inside `create_app()` silently degrades a body parameter into a query parameter (422)
The UI has no build step, so nothing catches a typo'd element id	`$("run-staus")` is `null.textContent` at runtime: the panel stops updating from that line on with nothing on screen to say why. A test cross-checks every id in `app.js` against `index.html`. **That test is the bundler**
BM25's IDF goes to zero, then negative, on a small corpus	A `score > 0` filter therefore drops a chunk that plainly contains the query tag — on an 11-page corpus, `PSLL-101` scored exactly 0.0. Membership is now decided by token overlap; BM25 is used only for *order*
A corrupt PDF does not raise `IngestError`	PyMuPDF raises its own types, so one truncated file aborted an eleven-document ingest and everything already loaded looked like the complete set
Document text is full of ρ, Δ, °	A cp1252 console raises `UnicodeEncodeError` and kills `kavach kb search` on exactly the corpus it exists to search. The ASCII-only house rule is about *our* strings; document text is data
A model is gated **twice**	`models.yaml` makes a model exist; `profiles.yaml` decides whether the router may consider it. Adding an entry and reloading is not enough — the id must be in the profile's `available:` list or it is never a candidate, with no error anywhere
`sse-starlette` needs its ping	Without `ping=`, an idle stream is dropped by the browser after 30–60 seconds and the panel goes dead with no error anywhere
On Windows, `GetIfTable2` lists every NIC up to six times	Each LightWeight Filter row repeats its adapter's *cumulative* totals, so summing the table multiplies egress by six and still looks like a measurement. The `FilterInterface` flag bit is the whole dedup

10. Hardware profiles — same code path, different box

Selected by one environment variable, `KAVACH_PROFILE`. No code change, no redesign — the same property that lets a better model be dropped in later.

Profile	Target	Budget	Context	Notes
TINY	CPU only	8.0 GB RAM	4,096	Contingency. Unpleasant but alive. Budgets against RAM because `vram_gb` is 0
LOW	RTX 4060, 8 GB	8.0 GB	8,192	2.0 GB reserved for fragmentation and display output
MID	RTX 3060, 12 GB	12.0 GB	16,384	**The reference profile.** 10.0 GB usable
HIGH	RTX 4080, 16 GB	16.0 GB	32,768	Two swap slots so vision never evicts text. The reranker is a swap-pool member, not pinned
ARC	Intel Arc iGPU	9.0 GB RAM	8,192	Shared system memory — there is no dedicated VRAM pool to budget against
FAST	6 GB laptop card	6.0 GB	8,192	Responsiveness over capability. Both models fit at once, so nothing is ever evicted and no request pays a swap — that, not raw tok/s, is most of the speed-up

Models in the registry

All open-weight, all Apache-2.0 or MIT, all quantised GGUF: Qwen2.5-7B-Instruct and Qwen3-8B (text, tools, reasoning), Qwen2.5-Coder-7B and -3B (code, fill-in-the-middle), Qwen2.5-VL-7B with its multimodal projector (vision — without the projector a vision model still loads, still answers, and never looks at the image, so the backend refuses images unless the projector is present), Qwen3-14B for the HIGH profile, Qwen2.5-1.5B and Qwen3-1.7B for TINY and FAST, plus bge-m3 and all-MiniLM-L6-v2 embedders and a bge-reranker-v2-m3. Thirteen GGUFs are staged on the development box, outside the repository, digest-verified at boot.

11. Verification and demo assets

789	0	67s	clean
TESTS PASSING	FAILURES	FULL SUITE	RUFF LINT

Tests are adversarial by convention — "a test that only asserts the happy path is not pulling its weight" — and are named for the property they defend, not the function they call. Alongside them sit runnable demo scripts, each of which answers one hostile question:

Script	Answers
`tamper_demo.py`	"How do you know the log hasn't been edited?" — breaks the chain five ways
`provenance_demo.py`	"How do you know those are the weights you approved?"
`swap_penalty_demo.py`	"What happens when two models don't fit on one card?"

`sandbox_demo.py` / `prove_sandbox.py`	"Can generated code reach the network or the filesystem?"
`eval_harness.py`	"Are those routing quality numbers measured or invented?"
`knowledge_demo.py`	"Does the answer actually come from the document?"
`offline_check.py`	"Does anything here phone home?" — asserts zero absolute URLs across the shipped UI
`run_sheet.py`	Generates the judge-facing status sheet, opening with what was **not** proved. Nothing in it is typed by hand — every line is read out of the live system, so it cannot drift between rehearsal and demo day

12. Current status and what remains

All seven phases are code-complete. **What remains is not code.** Everything so far has been validated against the mock backend, and six real-model checkpoints close that gap.

ID	What it measures	Needs
R0	Does `llama-server` spawn and reap correctly under the warden	1 GGUF
R1	Real `tok_per_s`, `load_ms` and measured VRAM, written back into `models.yaml`	same
R2	Routing accuracy on the *real* embedder (the 95.2% figure is a mock-embedder number)	bge-m3
R3	Measured `quality[model][class]` — closes the seeded-scores gate below	2–3 models
R4	**Tool-call reliability under GBNF, and whether a 7B model cites what it was given — the biggest single unknown in the build**	phases 4+5+6
R5	OCR/VLM on real scans — six of eleven MRPL documents are honest gaps until this runs	VL weights or tesseract

Two gates before anything goes in front of judges

> **1. Quality scores are hand-seeded**
>
> The router ranks on them and the panel shows them. Run `scripts/eval_harness.py --quality --write` against **real weights** before demoing the explainer panel. Until then every decision correctly carries `quality_provenance: "seeded"`.

> **2. Sovereignty is unproven on Windows**
>
> The canary needs `nftables` on a Linux box: `sudo nft -f deploy/nftables.conf`, then `kavach canary` → expect `BLOCKED` / `ENETUNREACH`. **This is the headline demo beat**, and it is the one thing a Windows laptop structurally cannot show.

Known limitations, stated plainly

- **RBAC is config-only.** The corpus panel has a role selector, and it is a dropdown the user picks — not a login. The clearance filter demonstrates the *mechanism*, not an authenticated identity.
- **Retrieval quality is unmeasured.** There is no labelled question set for the MRPL corpus, so "hybrid beats either arm" is demonstrated on two hand-picked queries, not measured. The router has an eval harness; the knowledge plane

has nothing equivalent yet.

- **No reranker stage.** `bge-reranker-v2-m3` is in the registry and in the HIGH profile's swap pool, and nothing calls it. RRF over two arms is the whole ranking.
- **The stream has no reconnect-with-cursor.** A browser that reconnects gets a fresh snapshot plus whatever the broker still buffers — correct, but it re-sends work. `Last-Event-ID` is the tidy fix and is not implemented.
- **One gateway process, one broker.** Run state is in SQLite and would survive, but the in-memory event buffer is not shared across workers. `--workers 1` is assumed, which is right for one box and worth knowing.
- **The panel is unstyled for a projector.** It was built for a laptop screen at one metre; the amber degradation banner in particular has to be legible from the back of a room.

Every status document opens with achievements and buries the caveats in a "future work" paragraph. The unproven list is the one a technical judge builds during Q&A anyway; handing it over up front is the difference between a caveat and a discovery — and it is what makes the proven half worth believing.